OPERATION SHADOWTRACE

CYBERSECURITY INCIDENT & DETECTION REPORT

A controlled purple-team lab demonstrating how suspicious activity can be generated, detected, and investigated using network and host-based security monitoring.

Prepared by	Ali Haider
Environment	Kali Linux attacker • Ubuntu target/monitor
Detection stack	Suricata IDS + Wazuh SIEM
Exercise window	30 October 2026 (lab evidence timestamps)
Classification	Educational lab report — sanitized copy

Privacy note: IP addresses, usernames and local host identifiers visible in the supplied screenshots have been obscured. No real-world external target was involved in the documented simulations.

1. Executive summary

Operation ShadowTrace is a controlled purple-team exercise. The attacker-side machine (Kali Linux) generated safe, limited test activity against a lab Ubuntu machine. Suricata inspected network traffic and generated alerts; Wazuh collected those alerts and host authentication events so they could be reviewed centrally.

Outcome

The lab demonstrated end-to-end detection for six scenarios: network scanning, SSH password-guessing attempts, ICMP discovery traffic, HTTP request activity, and repeated TCP connections used to emulate beacon-like command-and-control (C2) behavior. The supplied evidence PDF directly shows ICMP activity, Nmap SYN scanning, SSH brute-force alerting, and C2-related Wazuh alerts. TCP connect scanning and HTTP request detection were also verified during the lab workflow, but their screenshots are not in this upload.

6	SCENARIOS	2	CORE MONITORING TOOLS
Wazuh + Suricata	Integrated detection pipeline	Kali + Ubuntu	Controlled lab endpoints

Scope and safety

• All tests were run between lab systems under the project owner’s control.

• The C2 scenario was a repeated TCP connection simulation only; no malware, payload, or real command-and-control framework was deployed.

• Alert severity and rule IDs are reported as shown in the available evidence; a Suricata alert forwarded under Wazuh rule 86601 is not itself a severity-10 incident.

2. Lab architecture and data flow

The lab used two virtual machines on a host-only network. Kali generated test traffic; Ubuntu hosted the monitored services and the security stack. Suricata observed traffic on the host-only interface, wrote events to its EVE JSON log, and Wazuh ingested that log. Ubuntu authentication logs were also monitored by Wazuh for SSH failures.

KALI LINUX **Test / attacker role**	UBUNTU LINUX **Target + monitoring role**
Nmap • ping • SSH attempts HTTP request • TCP test connections	Suricata IDS → EVE JSON Wazuh Manager → alerts / Threat Hunting
Controlled lab traffic	Detection, collection and review

Key components

• Kali Linux: generated controlled reconnaissance and connection attempts.

• Ubuntu: served as the monitored host and ran Wazuh Manager and Suricata.

• Suricata: inspected packets and matched traffic against IDS rules.

• EVE JSON: structured event output written to /var/log/suricata/eve.json.

• Wazuh: collected Suricata events and SSH authentication logs for centralized alert review.

The private lab addresses are intentionally omitted from this report. The supplied screenshots have been sanitized.

3. Scenario findings

Each scenario below describes the activity in plain operational terms, what the monitoring stack observed, and the interpretation of the result.

01 — Nmap SYN scan

Activity: A SYN scan sends TCP connection-start packets to multiple ports to identify which ports may be open.

Detection and result: Suricata custom rule SID 1000004 fired with signature “LAB NMAP SYN scan detected”. The alert was forwarded to Wazuh under rule 86601. The evidence shows source and destination fields, TCP protocol, and a destination port.

ATT&CK: MITRE ATT&CK;: T1046 — Network Service Discovery.

Evidence status: Confirmed in supplied screenshots.

02 — SSH brute-force simulation

Activity: Repeated SSH login attempts using an intentionally incorrect password simulate password guessing against a remote account.

Detection and result: Ubuntu SSH authentication logs recorded failures. Wazuh generated rule 5760 authentication-failure events and rule 5763, “brute force trying to get access to the system”, at level 10 in the captured event.

ATT&CK: MITRE ATT&CK;: T1110 — Brute Force.

Evidence status: Confirmed in supplied screenshot.

03 — ICMP reconnaissance / ping

Activity: Twenty ping requests were sent to check whether the lab target was reachable. Repeated ICMP echo requests can be an early discovery signal.

Detection and result: The terminal evidence shows 20 packets transmitted, 20 received, and 0% packet loss. Suricata custom rule SID 1000005 detected the repeated ICMP activity; Wazuh displayed the corresponding Suricata alert.

ATT&CK: MITRE ATT&CK;: T1018 — Remote System Discovery (behavioral mapping for reachability discovery).

Evidence status: Confirmed in supplied screenshots.

04 — TCP connect scan

Activity: A TCP connect scan attempts to complete TCP connections to a range of ports. This can reveal services that accept connections.

Detection and result: The lab run used Nmap TCP connect scanning. The previously validated lab result showed Suricata SYN-related scan detection and forwarding into Wazuh. The current uploaded PDF does not contain a dedicated screenshot for this scenario.

ATT&CK: MITRE ATT&CK;: T1046 — Network Service Discovery.

Evidence status: Lab result previously verified; no dedicated screenshot in this upload.

05 — HTTP request detection

Activity: A client sends an HTTP request to a web server. Monitoring the request helps identify access patterns and the methods being used.

Detection and result: A HEAD request was sent with curl to the Ubuntu web service and returned HTTP 200. A custom Suricata HTTP rule (SID 1000006) generated an alert, which was visible in Wazuh. The uploaded PDF does not include the dedicated HTTP evidence screenshot.

ATT&CK: No malicious ATT&CK; technique is assigned to a normal HTTP HEAD request by itself.

Evidence status: Lab result previously verified; no dedicated screenshot in this upload.

06 — Simulated C2 beacon activity

Activity: The Kali machine repeatedly opened TCP connections to a listener on Ubuntu port 4444. Repeated callbacks can resemble a beacon pattern used by some malware.

Detection and result: Suricata rules SID 1000002 and 1000003 generated alerts for repeated and test connections. Wazuh received both under generic Suricata rule 86601. The listener confirmed an incoming connection from the lab peer. No malware or actual C2 protocol was used.

ATT&CK: ATT&CK; note: C2-like behavior was simulated. T1095 may be a conceptual analogy for non-application-layer communication, but the test does not establish a real adversary technique.

Evidence status: Confirmed in supplied screenshot(s) and prior lab validation.

4. Alert handling and interpretation

The project demonstrated two complementary detection paths: network-based detection through Suricata and host authentication monitoring through Wazuh. Suricata wrote structured alerts to EVE JSON. Wazuh read that file and displayed the events in Threat Hunting. Separately, Wazuh monitored Ubuntu SSH authentication logs and raised a higher-level brute-force alert after repeated failures.

Important alert interpretation

• Wazuh rule 86601, level 3: the generic Wazuh rule used to ingest/display Suricata alerts. It identifies a forwarded Suricata alert; it does not mean every event is a confirmed compromise.

• Wazuh rule 5763, level 10: the SSH brute-force detection shown in the evidence. This is a host authentication alert and is distinct from the Suricata network alerts.

• Suricata custom SIDs 1000002–1000006: lab rules authored for the project's C2-like TCP connections, scan/ICMP and HTTP detection cases.

Detection pipeline

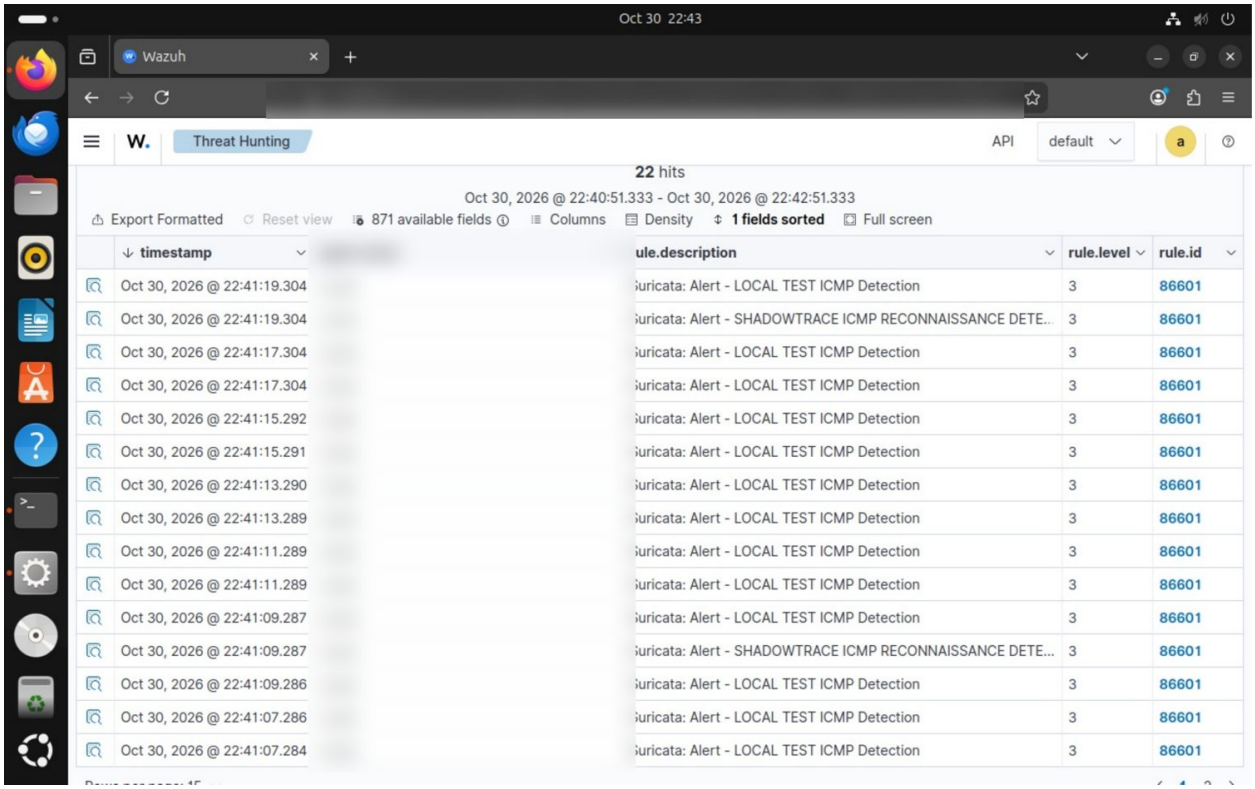
Kali activity → network traffic → Suricata rule match → EVE JSON event → Wazuh ingestion → analyst review in Threat Hunting. For SSH, authentication failures → Ubuntu auth log → Wazuh decoder/rule → brute-force alert.

Validation snapshot

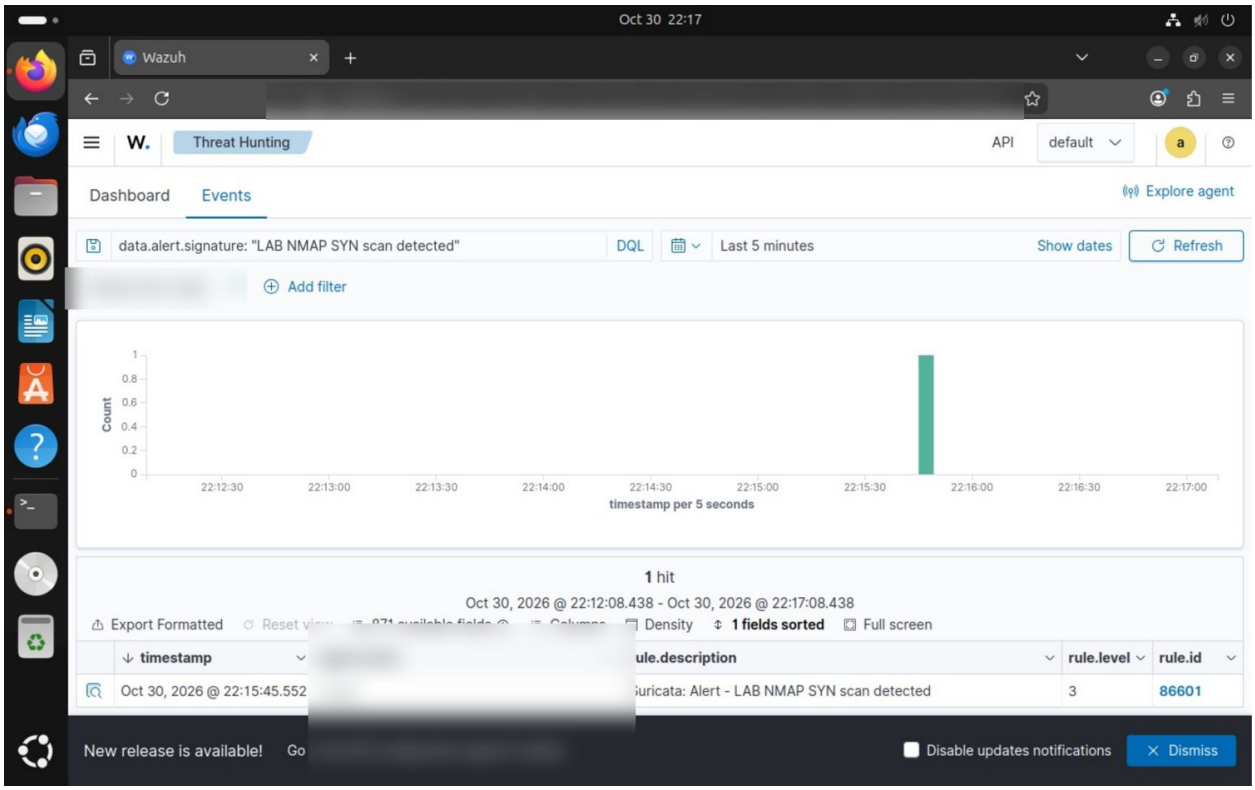
At final validation, both wazuh-manager and suricata returned "active". Ubuntu network interfaces were UP, with a separate NAT interface and host-only lab interface. The Kali-to-Ubuntu host-only connectivity had also been verified during the lab.

5. Evidence review

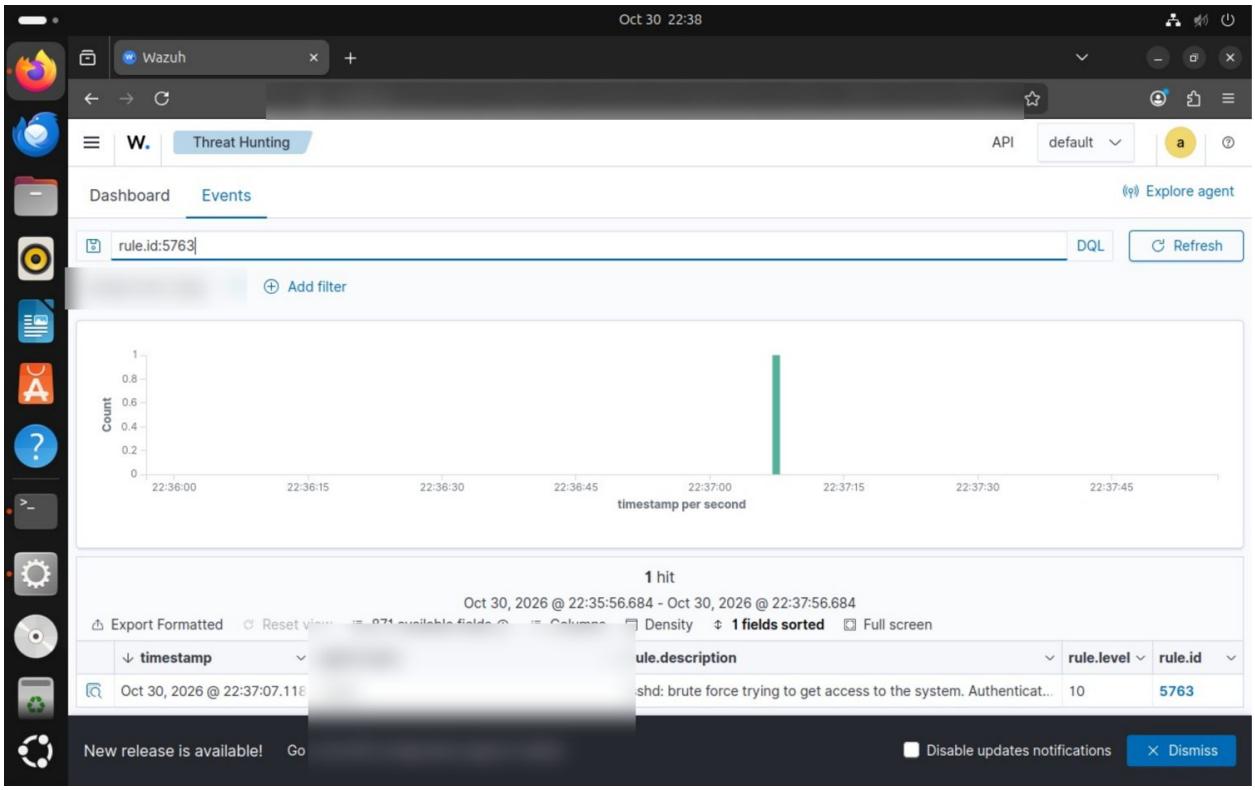
The following selected screenshots are taken from the nine-page evidence PDF supplied for this report. Repeated or redundant captures were excluded. Identifying details were blurred before embedding.



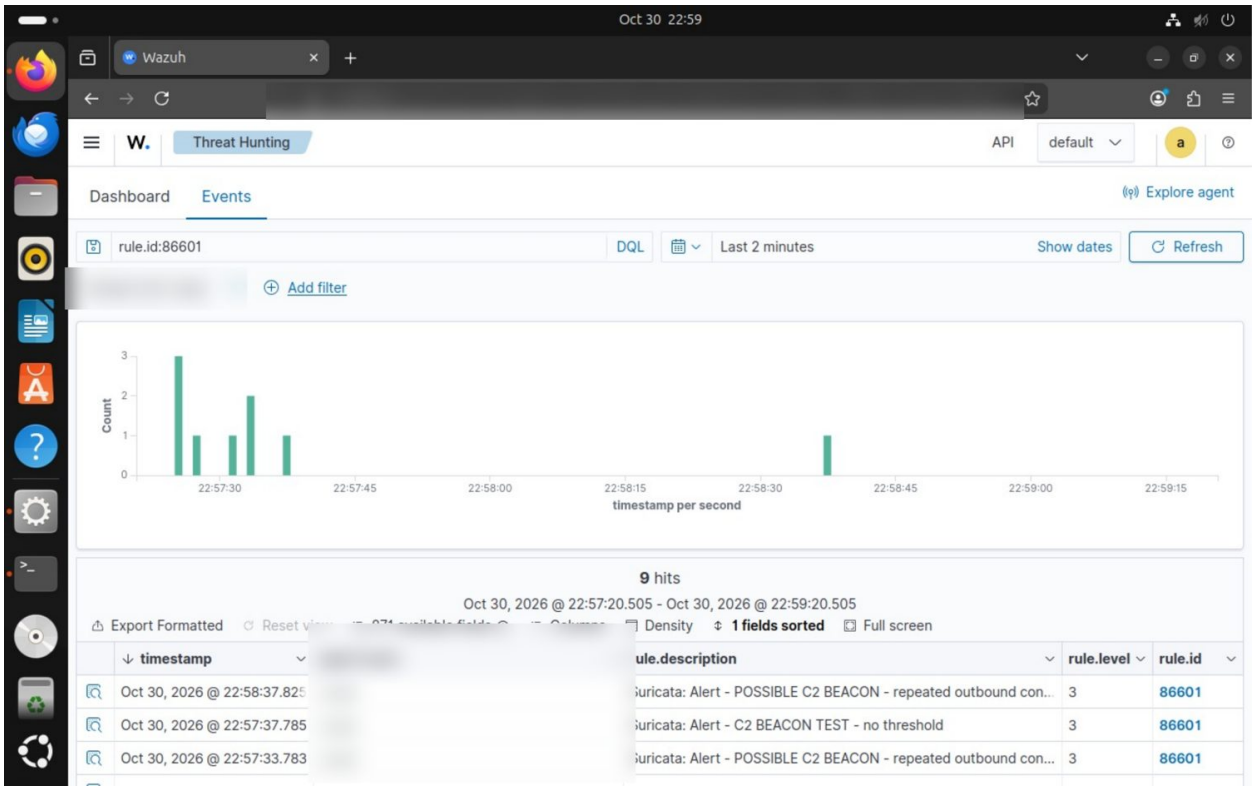
Evidence A — Wazuh Threat Hunting: Suricata ICMP reconnaissance alerts are visible among the event rows.



Evidence B — Wazuh Threat Hunting: the Nmap SYN scan alert is present under Wazuh rule 86601.



Evidence C — Wazuh Threat Hunting: SSH brute-force rule 5763 appears at level 10.



Evidence D — Wazuh Threat Hunting: C2 beacon test and repeated-connection alert descriptions are visible.

6. Results, limitations and recommendations

Results achieved

• Established a working Suricata IDS sensor on the host-only lab interface.

• Forwarded Suricata EVE JSON alerts into Wazuh for centralized review.

• Detected Nmap scan activity, repeated ICMP requests, HTTP request activity, and repeated TCP connections used for the C2-beacon simulation.

• Detected SSH authentication failures and a brute-force pattern through Wazuh.

• Validated that Wazuh Manager and Suricata services were active at project closeout.

Limitations and careful claims

• This was a small, controlled lab, not a production SOC deployment. Results do not establish detection coverage against all tools or attacker techniques.

• The C2 test used repeated TCP connections to a local listener; it did not run malware, an implant, or a real C2 framework.

• The ordinary HTTP HEAD request was benign. The custom rule demonstrated request visibility, not malicious intent.

• The supplied evidence PDF did not include dedicated screenshots for TCP connect scanning or HTTP request detection. Those outcomes are included because they were verified in the lab workflow, but are not presented as screenshot-backed in this report.

• Some screenshots show duplicate local ICMP test alerts; redundant rows were not treated as separate scenarios.

Recommended next steps

• Retain sanitized raw EVE JSON and Wazuh alert exports for each scenario, with timestamps and rule IDs.

• Add scenario-specific Wazuh rules and meaningful severity tuning instead of relying only on generic Suricata ingestion rule 86601.

• Document false-positive considerations and test detection thresholds with both normal and suspicious traffic.

• Keep lab accounts, listeners, and test services disabled when the exercise is not running.

Conclusion

Operation ShadowTrace demonstrated the core SOC workflow: generate controlled activity, observe it at the network or host level, detect it with security rules, centralize the alerts, and interpret what the evidence does—and does not—prove. The project provides a practical foundation for further SOC analyst training.

Prepared by Ali Haider | Operation ShadowTrace | Sanitized educational incident report